

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**

 Bug **15**

 Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

- Generic function type aliases should be preferred


- Type parameters should not shadow other type parameters


- "static final" declarations should be "const" instead


- Interpolation should be used instead of String concatenation


- Collection literals should be preferred


- Conditional assignment should be preferred


- "this" should only be used when required


- Exceptions should not be ignored


- Code annotated as deprecated should not be used


- File names should comply with a naming convention


- Unused local variables should be removed


- Package names should comply with a naming convention

Generic function type aliases should be preferred

Analyze your code

Consistency - Conventional

Maintainability ▾

 Code Smell  Minor ⓘ  cppcoreguidelines design since-c++11

Why is this an issue?

More Info

In Dart type aliases can be declared via function type aliases `typedef void F()` or generic function type aliases `typedef F = void Function()`. Generic function type aliases can be parametrized `typedef Compare<T> = int Function(T a, T b);`. Function type aliases miss this feature.

While it's not always needed to have them parametrized, for the sake of readability it's recommended to use consistent way of declaring type aliases.

Thus, generic function type aliases should be preferred.

Noncompliant code example

```
typedef void F(int a);
```

Compliant solution

```
typedef F = void Function(int a)
```

Available In:
sonarcloud 

